
Sky130 Full Chip Template

Lawrence Roman A. Quizon

November 2025

OpenROAD Pading Generation

1. Write Verilog
2. Write OpenROAD script
 1. Generate Rows
 2. Place Pads
 3. Place Fill Pads
 4. Place Corners
 5. Connect by Abutment
 6. Place IO Pins
 7. Write DEF
3. Magic reads DEF and writes out GDS

Write Pading Verilog

- Declare the module
- PG connections are **inout**
- Pad is physical pad
- There are other inputs from core-side to configure the pads.

```
3
4  module chip_io (
5      // Power and ground connections
6      inout vccd,
7      // ...
8      inout vssio,
9
10     // GPIO
11     inout [numGPIO-1:0] gpio_pad,
12     output [numGPIO-1:0] gpio_in,
13     input [numGPIO-1:0] gpio_out
14 )
15 // ...
16
```

Write Pading Verilog

- I recommend to declare many pads as a generate block

```
// GPIO input pads
genvar i;
generate
    for (i = 0; i < numGPIO; i = i + 1) begin : gpio_pads
        sky130_ef_io__gpiov2_pad_wrapped u_gpio_pad (
            // ...
            .DM          ({ dm2[i], dm1[i], dm0[i] }), // From core
            // ...
            .VSWITCH      (vddio),
            .VTRIP_SEL     (vssd) // From core
        );
    end
endgenerate
```

Write Pading Verilog

- Power pads in SKY130 all have different master, so declare them separately.

```
// Power Pads
sky130_ef_io__vccd_lvc_pad u_vccd_pad (
    // ...
);
sky130_ef_io__vssd_lvc_pad u_vssd_pad (
    // ...
);
sky130_ef_io__vddio_hvc_pad u_vddio_pad (
    // ...
);
sky130_ef_io__vssio_hvc_pad u_vssio_pad (
    // ...
);
```

endmodule

Write Openroad TCL Script

- The OpenROAD script reads the Verilog in order to determine which connections are supposed to exist.
- You can find this in `or_chip_io/openroad.tcl`
- At the start of the TCL sources all the standard LEFs

```
# Read tech lef
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/techlef/sky130_fd_sc_hd__nom.tlef

# Read standard cell lefs
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/lef/sky130_ef_sc_hd.lef
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/lef/sky130_fd_sc_hd.lef

# Read pad lefs
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_io/lef/sky130_fd_io.lef
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_io/lef/sky130_ef_io.lef

# Read the nominal corner timing library
read_liberty /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib
```

OR TCL: Tech LEFs

- The OpenROAD script reads the Verilog in order to determine which connections are supposed to exist.
- At the start, the TCL sources all the standard LEFs

```
# Read tech lef
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/techlef/sky130_fd_sc_hd__nom.tlef

# Read standard cell lefs
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/lef/sky130_ef_sc_hd.lef
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/lef/sky130_fd_sc_hd.lef

# Read pad lefs
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_io/lef/sky130_fd_io.lef
read_lef /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_io/lef/sky130_ef_io.lef

# Read the nominal corner timing library
read_liberty /foss/pdks/ciel/sky130/versions/0536d02d875c8f67dd7cca3902ac457e62f20005/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib
```

OR TCL: Floorplan

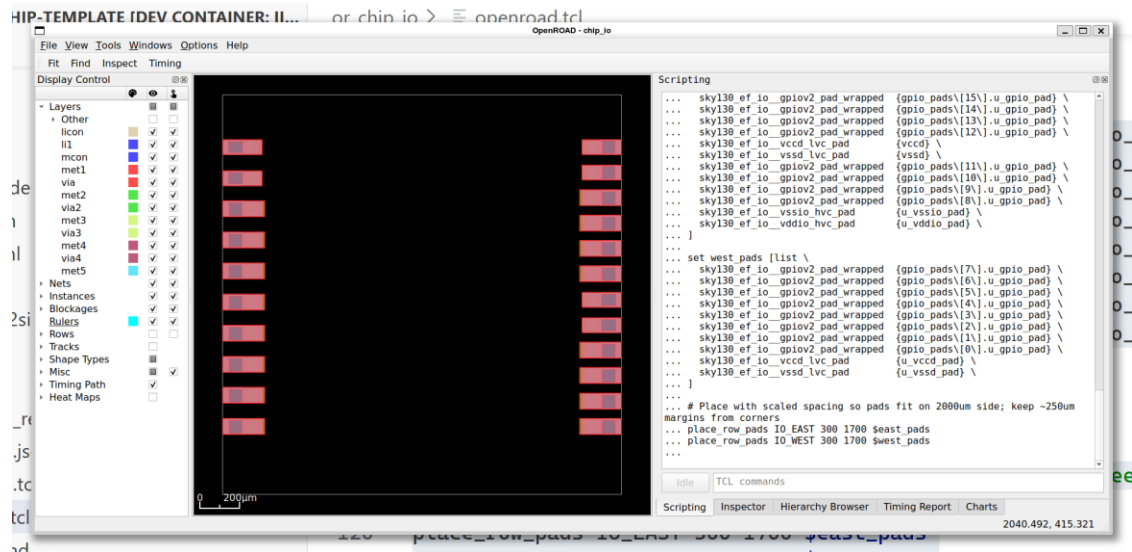
- Then, we read the Verilog and declare a die area and core area.
- The padding is about ~250um margin. Openroad later adjusts the placement accordingly to be exact.

```
read_verilog /workspaces/sky130-full-chip-template/or_chip_io/outputs/chip_io.gl.v
link_design chip_io
```

```
initialize_floorplan \  
-die_area "0 0 2000 2000" \  
-core_area "250 250 1750 1750" \  
-site unithd
```


OR TCL: Place Pads

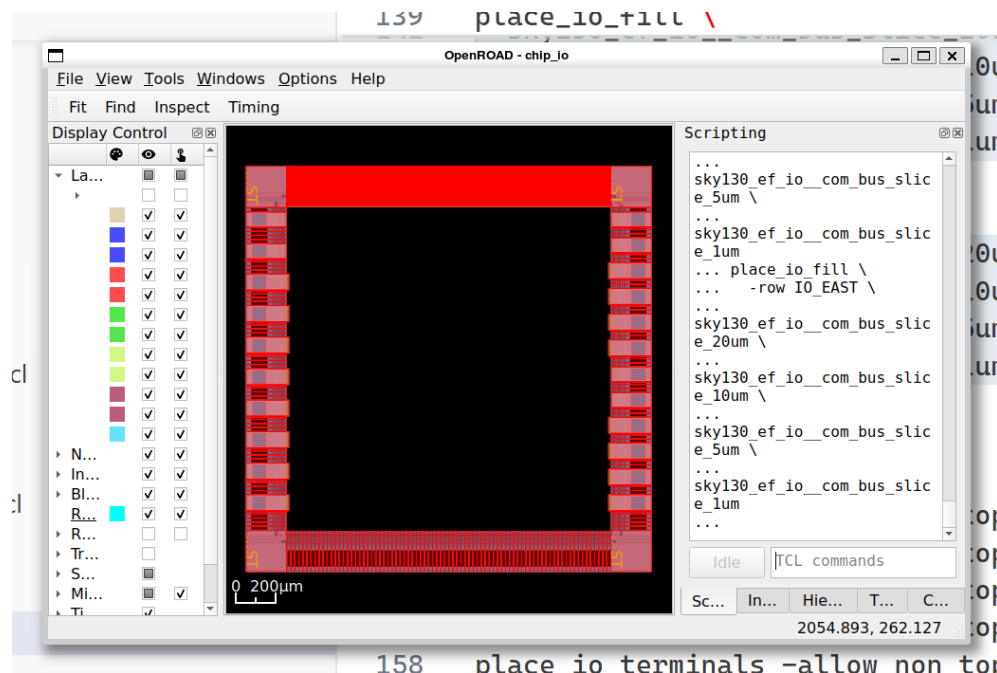
- `place_pad` commands OpenROAD to actually place down the pads at a specific offset from the left of the tracks
- i.e. Offset is from WEST if row is IO_NORTH, SOUTH if row is IO_WEST



```
##### Place Pads #####
place_pad \
  -master
  sky130_ef_io__connect_vcchib_vccd_and_vswitch_vddio_slice_20um
  \
  -row IO_SOUTH \
  -location 257.0 \
  {connect_0}
# ...
```

OR TCL: Corners and Fill

- `place_corners <corner master model>`
- `place_io_fill -row (ROW) [<io fill cells list>]`
- OpenROAD will automatically choose and use which fills are needed to fill completely.

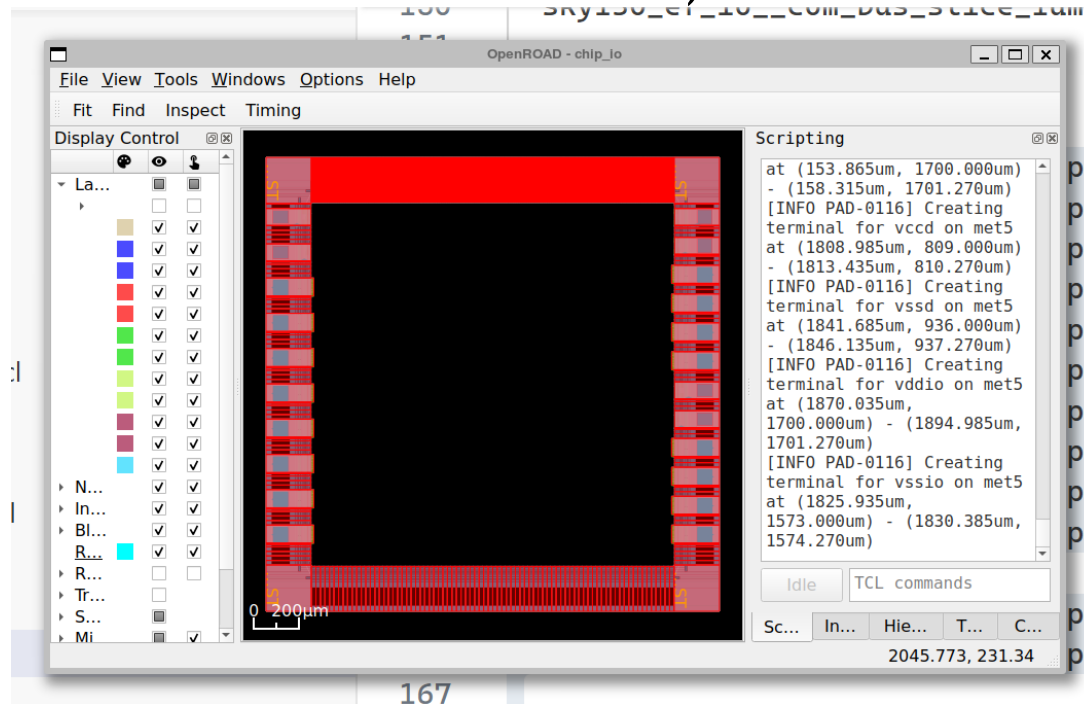


```
# Place corner
place_corners sky130_ef_io__corner_pad

# Place fill
place_io_fill \
  -row IO_NORTH \
  sky130_ef_io__com_bus_slice_1um \
  sky130_ef_io__com_bus_slice_5um \
  sky130_ef_io__com_bus_slice_20um \
  sky130_ef_io__com_bus_slice_10um
place_io_fill \
  -row IO_SOUTH \
  sky130_ef_io__com_bus_slice_20um \
  sky130_ef_io__com_bus_slice_10um \
  sky130_ef_io__com_bus_slice_5um \
  sky130_ef_io__com_bus_slice_1um
place_io_fill \
  -row IO_WEST \
  sky130_ef_io__com_bus_slice_20um \
  sky130_ef_io__com_bus_slice_10um \
  sky130_ef_io__com_bus_slice_5um \
  sky130_ef_io__com_bus_slice_1um
place_io_fill \
  -row IO_EAST \
  sky130_ef_io__com_bus_slice_20um \
  sky130_ef_io__com_bus_slice_10um \
  sky130_ef_io__com_bus_slice_5um \
  sky130_ef_io__com_bus_slice_1um
```

OR TCL: Connect by Abutment and IO

- `connect_by_abutment` doesn't do anything physical, but tells OpenROAD to logically connect nets that are abutted in the padring.
- `place_io_terminals` adds pins that are declared in the DEF (that the core can later use to connect)



```
connect_by_abutment
place_io_terminals */PAD
place_io_terminals -allow_non_top_layer */IN
place_io_terminals -allow_non_top_layer */HLD_OVR
place_io_terminals -allow_non_top_layer */IB_MODE_SEL
place_io_terminals -allow_non_top_layer */INP_DIS
place_io_terminals -allow_non_top_layer */OE_N
place_io_terminals -allow_non_top_layer */OUT
place_io_terminals -allow_non_top_layer */PAD
place_io_terminals -allow_non_top_layer */DM\[0\]
place_io_terminals -allow_non_top_layer */DM\[1\]
place_io_terminals -allow_non_top_layer */DM\[2\]
```

OR TCL: Write DEF

- DEF file
 - Contains components, locations, and pin names.
- **MAGIC** reads the DEF file and writes a GDS

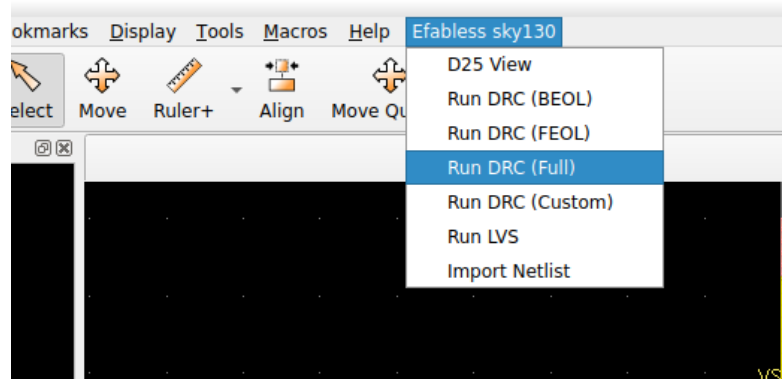
```
set def_file "chip_io.def"  
write_def $def_file
```

```
1  VERSION 5.8 ;  
2  DIVIDERCHAR "/" ;  
3  BUSBITCHARS "[" ;  
4  DESIGN chip_io ;  
5  UNITS DISTANCE MICRONS 1000 ;  
6  DIEAREA ( 0 0 ) ( 2000000 2000000 ) ;  
7  // Rows are here  
8  > COMPONENTS 1831 ; ...  
1840  END COMPONENTS  
1841  > PINS 150 ; ...  
2634  END PINS  
2635  > SPECIALNETS 154 ; ...  
5556  END SPECIALNETS  
5557  > NETS 16 ; ...  
5574  END NETS  
5575  END DESIGN  
5576  |
```

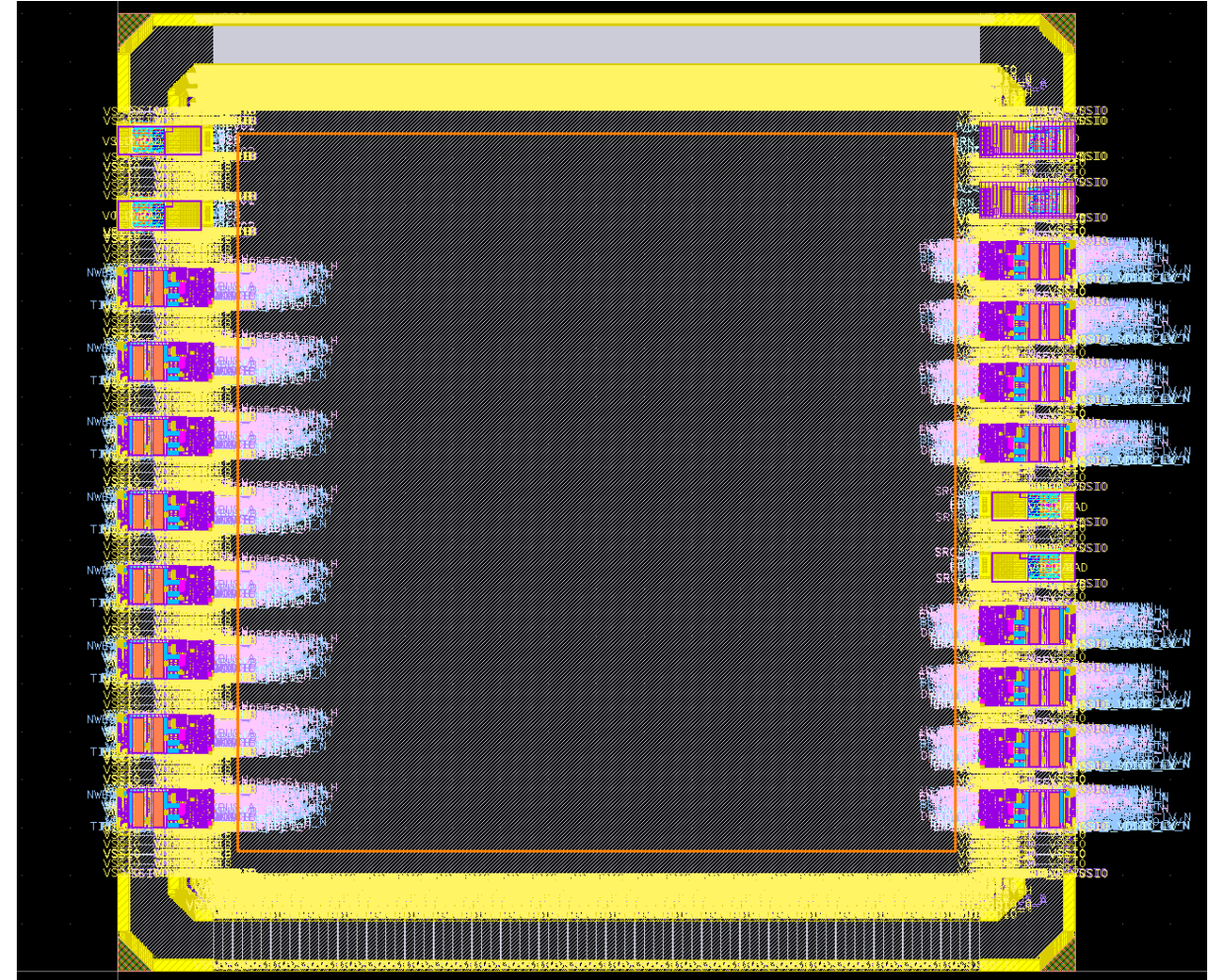
```
1  def read outputs/chip_io.def  
2  drc check  
3  drc count  
4  drc listall why  
5  gds write outputs/chip_io.gds  
6  quit
```


Padring Final GDS and DRC

- DRC in KLayout can be done via GUI



- But there's also a script we can use in
`$PDKPATH/libs.tech/klayout/klayout_gds_drc_check.py`



LVS of Padring

- We use NetGen to do this.
- **MAGIC** (via magic_ext2sim.tcl) extracts the layout netlist (extract/chip_io.spice)
- **NETGEN** compares the gate-level (outputs/chip_io.gl.v) and the extracted (extract/chip_io.spice)

```
# Extract SPICE
$(EXT_DIR)/$(TOP).spice: magic_ext2sim.tcl | $(EXT_DIR)
    $(MAGIC) -dnull -noconsole -rcfile $(MAGIC_RC_FILE) $<

# LVS
.PHONY: lvs
lvs:    $(REPORTS_DIR)/netgen_lvs.rpt
$(REPORTS_DIR)/netgen_lvs.rpt: netgen_lvs.tcl | $(REPORTS_DIR)
    $(NETGEN) -batch source $<
```

Padring Simulation

- Usual way
 - Instantiate pad Verilog in a chip_top.sv along with the core
 - Write a testbench that stimulates the PAD input.
- However
 - Sky130 pad Verilog simulation infeasible using OS tools
- **sky130_fd_io cannot be simulated by OS tools**
 - Verilator cannot handle transistor-level declarations
 - IVerilog cannot handle “cheating” hierarchical references
- Alternative: **Simulate full-spice.**

Padring Simulation

- `sim_with_pad` shows a potential flow to do this with Verilog
- `tb_regfile_top` produces the stimulus for the physical pads
- `chip_top.sv` instantiates the padring and the regfile (core).
- `patch_sky130_fd_io.py` was an attempt at fixing the sky130_fd_io Verilog models to be compatible with IVerilog.
- See the **Makefile** for the general flow

```
sim_with_pad/  
├── Makefile  
├── README.md  
├── scripts  
│   └── patch_sky130_fd_io.py  
├── src  
│   ├── chip_io.sv  
│   ├── chip_top.sv  
│   └── regfile.sv  
└── tb  
    └── tb_regfile_top.sv
```


Schematic Tool for Padring

- Unusual
 - At least no previous Microlab tapeout used schematic editors for padring
 - No OS tool uses schematic editors to generate the padring
- **Can use schematic editors to simulate the pads spice-level, but it might be easier to just go straight to ngspice.**

Padring Timing Analysis

- We use STA to do reliable padring timing analysis
- For the core, An SDC file is used to declare
 - A clock with jitter
 - I/O delays
 - With pads, CLK typically has the same delay as the IO so it doesn't affect the timing arcs.
- Then, overall, STA can be applied to chip_top.sv to include the pad timing libraries

```
set CLK_PERIOD 20.0
```

```
create_clock -name clk -period $CLK_PERIOD  
[get_ports clk]
```

```
set_clock_uncertainty 1.0 [get_clocks clk]
```

```
# Input delay = 20% of 20ns = 4ns  
set_input_delay [expr 0.2 * $CLK_PERIOD] -  
clock clk [all_inputs]
```

```
# Output delay = 30% of 20ns = 6ns  
set_output_delay [expr 0.3 * $CLK_PERIOD] -  
clock clk [all_outputs]
```

Padding Timing Analysis

- STA not usually done including padings. Typically just core, then the padings are accounted for via the SDC I/O settings for the core.

```
set CLK_PERIOD 20.0
```

```
create_clock -name clk -period $CLK_PERIOD  
[get_ports clk]
```

```
set_clock_uncertainty 1.0 [get_clocks clk]
```

```
# Input delay = 20% of 20ns = 4ns
```

```
set_input_delay [expr 0.2 * $CLK_PERIOD] -  
clock clk [all_inputs]
```

```
# Output delay = 30% of 20ns = 6ns
```

```
set_output_delay [expr 0.3 * $CLK_PERIOD] -  
clock clk [all_outputs]
```